

## Exp 9

```
import numpy as np
import matplotlib.pyplot as plt
from keras.models import Sequential
from keras.layers import Dense
```

```
# Prepare the dataset for AND gate
X_and = np.array([[0, 0],
[0, 1],
[1, 0],
[1, 1]])
y_and = np.array([[0], [0], [0], [1]]) # AND gate outputs
```

```
# Prepare the dataset for NAND gate
X_nand = np.array([[0, 0],
[0, 1],
[1, 0],
[1, 1]])
y_nand = np.array([[1], [1], [1], [0]]) # NAND gate outputs
```

```
# Function to create and train the model
def create_and_train_model(X, y):
    # Create the model
    model = Sequential()
    model.add(Dense(2, input_dim=2, activation='sigmoid')) # Hidden layer
    model.add(Dense(1, activation='sigmoid')) # Output layer

    # Compile the model
    model.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])

    # Train the model
    model.fit(X, y, epochs=5000, verbose=0)
    return model
```

```
# Train the AND gate model
and_model = create_and_train_model(X_and, y_and)
print("AND Gate Predictions:")
print(and_model.predict(X_and))
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106: UserWarning: Do not pass an `input_shape` to `input`
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
AND Gate Predictions:
1/1  0s 57ms/step
[[0.01465188]
 [0.08377758]
 [0.08454863]
 [0.8120682 ]]
```

```
# Train the NAND gate model
nand_model = create_and_train_model(X_nand, y_nand)
print("\nNAND Gate Predictions:")
print(nand_model.predict(X_nand))
```

```
NAND Gate Predictions:
1/1  0s 57ms/step
[[0.98754555]
 [0.93355346]
 [0.9363209 ]
 [0.1445951 ]]
```

```
# Visualize predictions
def plot_predictions(model, X, title):
    predictions = model.predict(X)
    plt.figure()
    plt.scatter(X[:, 0], X[:, 1], c=predictions.flatten(), cmap='coolwarm', s=100)
    plt.title(title)
    plt.xlabel('Input 1')
    plt.ylabel('Input 2')
    plt.colorbar(label='Output')
    plt.xlim(-0.5, 1.5)
    plt.ylim(-0.5, 1.5)
    plt.axhline(0.5, color='grey', lw=0.5, ls='--')
    plt.axvline(0.5, color='grey', lw=0.5, ls='--')
    plt.show()
```

```
# Plotting the predictions for both gates
plot_predictions(and_model, X_and, "AND Gate Predictions")
plot_predictions(nand_model, X_nand, "NAND Gate Predictions")
```

